

DD2440 Advanced Algorithms
Fall 2026
Lecture 2: Approx Algorithms (cont.)

Ioana-Oriana Bercea

2025-09-9

Agenda

Vertex Cover

Max-Cut

k -Center

Agenda

Vertex Cover

Max-Cut

k -Center

Approximation Algorithm for Minimum Vertex Cover

Input: Graph $G = (V, E)$

Output: A vertex cover $S \subseteq V$ of G

Goal: Minimize size of S

```
1 function MinVC( $G$ ):  
2   Let  $S = \emptyset$   
3   while exists an edge  $e = (u, v)$  such that  $u \notin S$  and  $v \notin S$   
4     |  $S \leftarrow S \cup \{u, v\}$   
5   return  $S$ 
```

Analyzing the Algorithm - intuition roadmap

- ▶ Step 0: Prove **correctness**: the **Alg** indeed returns a vertex cover

Analyzing the Algorithm - intuition roadmap

- ▶ Step 0: Prove **correctness**: the **Alg** indeed returns a vertex cover
- ▶ think about what **Alg** does vs. what an optimal solution does
- ▶ Step -1: Establish some **notation**: let's say **Alg** outputs S and an optimal solution is some set S^*

Analyzing the Algorithm - intuition roadmap

- ▶ Step 0: Prove **correctness**: the **Alg** indeed returns a vertex cover
- ▶ think about what **Alg** does vs. what an optimal solution does
- ▶ Step -1: Establish some **notation**: let's say **Alg** outputs S and an optimal solution is some set S^*
- ▶ Step 1: **Understand** what you're trying to prove:

Analyzing the Algorithm - intuition roadmap

- ▶ Step 0: Prove **correctness**: the **Alg** indeed returns a vertex cover
- ▶ think about what **Alg** does vs. what an optimal solution does
- ▶ Step -1: Establish some **notation**: let's say **Alg** outputs S and an optimal solution is some set S^*
- ▶ Step 1: **Understand** what you're trying to prove:
 - ▶ $|S| \leq 2 \cdot |S^*| \leftarrow$ what does this even mean?

Analyzing the Algorithm - intuition roadmap

- ▶ Step 0: Prove **correctness**: the **Alg** indeed returns a vertex cover
- ▶ think about what **Alg** does vs. what an optimal solution does
- ▶ Step -1: Establish some **notation**: let's say **Alg** outputs S and an optimal solution is some set S^*
- ▶ Step 1: **Understand** what you're trying to prove:
 - ▶ $|S| \leq 2 \cdot |S^*| \leftarrow$ what does this even mean?
 - ▶ **Alg** should not choose many more vertices than **Opt**
 - ▶ what is an **extreme** case?

Analyzing the Algorithm - intuition roadmap

- ▶ Step 0: Prove **correctness**: the **Alg** indeed returns a vertex cover
- ▶ think about what **Alg** does vs. what an optimal solution does
- ▶ Step -1: Establish some **notation**: let's say **Alg** outputs S and an optimal solution is some set S^*
- ▶ Step 1: **Understand** what you're trying to prove:
 - ▶ $|S| \leq 2 \cdot |S^*| \leftarrow$ what does this even mean?
 - ▶ **Alg** should not choose many more vertices than **Opt**
 - ▶ what is an **extreme** case? **Alg** picks "unnecessary" vertices???

Analyzing the Algorithm - intuition roadmap

- ▶ Step 0: Prove **correctness**: the **Alg** indeed returns a vertex cover
- ▶ think about what **Alg** does vs. what an optimal solution does
- ▶ Step -1: Establish some **notation**: let's say **Alg** outputs S and an optimal solution is some set S^*
- ▶ Step 1: **Understand** what you're trying to prove:
 - ▶ $|S| \leq 2 \cdot |S^*| \leftarrow$ what does this even mean?
 - ▶ **Alg** should not choose many more vertices than **Opt**
 - ▶ what is an **extreme** case? **Alg** picks "unnecessary" vertices???
- ▶ Step 2: **Defend** the **Alg**
 - ▶ all edges **Alg** looks at must be covered, so I **have** to pick at least one vertex from each!
 - ▶ this doesn't feel "unnecessary"... but why?

Analyzing the Algorithm - intuition roadmap

- ▶ Step 0: Prove **correctness**: the **Alg** indeed returns a vertex cover
- ▶ think about what **Alg** does vs. what an optimal solution does
- ▶ Step -1: Establish some **notation**: let's say **Alg** outputs S and an optimal solution is some set S^*
- ▶ Step 1: **Understand** what you're trying to prove:
 - ▶ $|S| \leq 2 \cdot |S^*| \leftarrow$ what does this even mean?
 - ▶ **Alg** should not choose many more vertices than **Opt**
 - ▶ what is an **extreme** case? **Alg** picks "unnecessary" vertices???
- ▶ Step 2: **Defend** the **Alg**
 - ▶ all edges **Alg** looks at must be covered, so I **have** to pick at least one vertex from each!
 - ▶ this doesn't feel "unnecessary"... but why?
 - ▶ **Opt** has to pick at least one vertex from every edge I look at

Analyzing the Algorithm - general formal structure

Remember: $|S| \leq 2 \cdot |S^*| \leftarrow$ this is what we're trying to prove

Analyzing the Algorithm - general formal structure

Remember: $|S| \leq 2 \cdot |S^*| \leftarrow$ this is what we're trying to prove

*"all edges **Alg** looks at must be covered, so I **have** to pick at least one vertex from each!"*

Analyzing the Algorithm - general formal structure

Remember: $|S| \leq 2 \cdot |S^*| \leftarrow$ this is what we're trying to prove

*"all edges **Alg** looks at must be covered, so I **have** to pick at least one vertex from each!"*

- ▶ say A is the set of edges the **Alg** looks at (give names)
- ▶ I pick twice as many vertices

$$|S| = 2 \cdot |A|$$

Analyzing the Algorithm - general formal structure

Remember: $|S| \leq 2 \cdot |S^*| \leftarrow$ this is what we're trying to prove

*"all edges **Alg** looks at must be covered, so I **have** to pick at least one vertex from each!"*

- ▶ say A is the set of edges the **Alg** looks at (give names)
- ▶ I pick twice as many vertices

$$|S| = 2 \cdot |A| \leftarrow \text{okay but prove this (later)}$$

Analyzing the Algorithm - general formal structure

Remember: $|S| \leq 2 \cdot |S^*| \leftarrow$ this is what we're trying to prove

*"all edges **Alg** looks at must be covered, so I **have** to pick at least one vertex from each!"*

- ▶ say A is the set of edges the **Alg** looks at (give names)
- ▶ I pick twice as many vertices

$$|S| = 2 \cdot |A| \leftarrow \text{okay but prove this (later)}$$

*"**Opt** has to pick at least one vertex from every edge I look at"*

Analyzing the Algorithm - general formal structure

Remember: $|S| \leq 2 \cdot |S^*| \leftarrow$ this is what we're trying to prove

*"all edges **Alg** looks at must be covered, so I **have** to pick at least one vertex from each!"*

- ▶ say A is the set of edges the **Alg** looks at (give names)
- ▶ I pick twice as many vertices

$$|S| = 2 \cdot |A| \leftarrow \text{okay but prove this (later)}$$

*"**Opt** has to pick at least one vertex from every edge I look at"*

$$|A| \leq |S^*|$$

Analyzing the Algorithm - general formal structure

Remember: $|S| \leq 2 \cdot |S^*| \leftarrow$ this is what we're trying to prove

*"all edges **Alg** looks at must be covered, so I **have** to pick at least one vertex from each!"*

- ▶ say A is the set of edges the **Alg** looks at (give names)
- ▶ I pick twice as many vertices

$$|S| = 2 \cdot |A| \leftarrow \text{okay but prove this (later)}$$

*"**Opt** has to pick at least one vertex from every edge I look at"*

$$|A| \leq |S^*| \leftarrow \text{okay but prove this (later)}$$

Analyzing the Algorithm - general formal structure

Remember: $|S| \leq 2 \cdot |S^*| \leftarrow$ this is what we're trying to prove

*"all edges **Alg** looks at must be covered, so I **have** to pick at least one vertex from each!"*

- ▶ say A is the set of edges the **Alg** looks at (give names)
- ▶ I pick twice as many vertices

$$|S| = 2 \cdot |A| \leftarrow \text{okay but prove this (later)}$$

*"**Opt** has to pick at least one vertex from every edge I look at"*

$$|A| \leq |S^*| \leftarrow \text{okay but prove this (later)}$$

Take it home:

$$|S| = 2 \cdot |A| \leq 2 \cdot |S^*|$$

Analyzing the Algorithm - general formal structure (1)

$$|S| = 2 \cdot |A|$$

- ▶ not trivial!
- ▶ first thought: well, for every edge in A , there are 2 vertices I pick (this is **Alg**)
- ▶ but this just proves that $|S| \leq 2 \cdot |A|$
- ▶ possible problem: I pick the same vertex twice
- ▶ so you also want to prove that $|S| \geq 2 \cdot |A|$
- ▶ you do this by arguing that the set of edges in A do not share any vertices $\leftarrow$ super important

Analyzing the Algorithm - general formal structure (2)

$$|A| \leq |S^*|$$

- ▶ maybe harder but maybe not?
- ▶ for every edge $(u, v) \in A$, either u or v have to be in S^*
- ▶ easiest by contradiction: suppose otherwise
- ▶ then there must exist an edge in A where neither u nor v have been chosen

Analyzing the Algorithm - general formal structure (2)

$$|A| \leq |S^*|$$

- ▶ maybe harder but maybe not?
- ▶ for every edge $(u, v) \in A$, either u or v have to be in S^*
- ▶ easiest by contradiction: suppose otherwise
- ▶ then there must exist an edge in A where neither u nor v have been chosen $\leftarrow$ because the edges in A don't share any vertices

Analyzing the Algorithm - general formal structure (2)

$$|A| \leq |S^*|$$

- ▶ maybe harder but maybe not?
- ▶ for every edge $(u, v) \in A$, either u or v have to be in S^*
- ▶ easiest by contradiction: suppose otherwise
- ▶ then there must exist an edge in A where neither u nor v have been chosen $\leftarrow$ because the edges in A don't share any vertices
- ▶ why is this bad? Because if such an edge exists, then S^* is not a vertex cover

Agenda

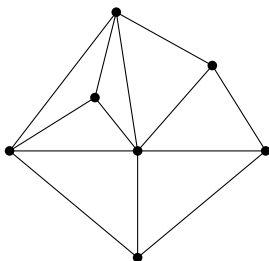
Vertex Cover

Max-Cut

k -Center

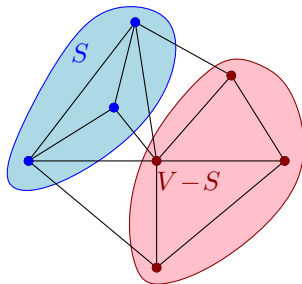
Cuts in graphs

A cut in a graph is a partition of vertices into two parts



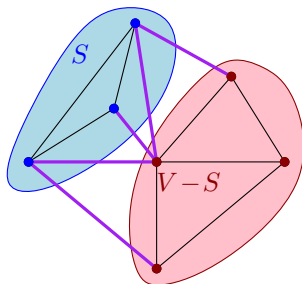
Cuts in graphs

A cut in a graph is a partition of vertices into two parts



Cuts in graphs

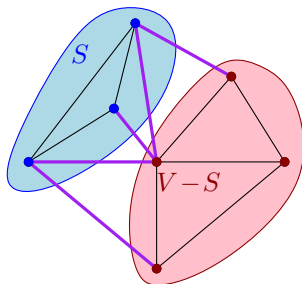
A cut in a graph is a partition of vertices into two parts



The **size** of the cut $|E(S, V - S)|$ is the *number of edges between the two parts*.

Cuts in graphs

A cut in a graph is a partition of vertices into two parts

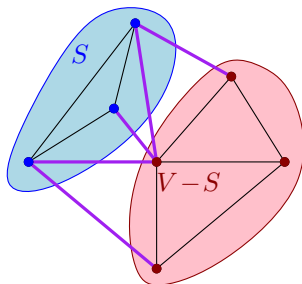


The **size** of the cut $|E(S, V - S)|$ is the *number of edges between the two parts*.

Many natural problems we may want to pursue:
make small cut, make large cut, S half the vertices, etc...

Cuts in graphs

A cut in a graph is a partition of vertices into two parts



The **size** of the cut $|E(S, V - S)|$ is the *number of edges between the two parts*.

Many natural problems we may want to pursue:
make small cut, **make large cut**, S half the vertices, etc...

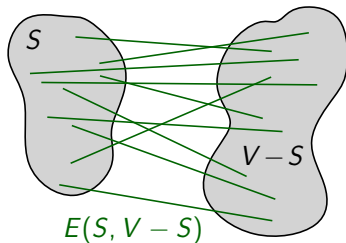
Max-Cut

The MAX-CUT problem:

Input: Graph $G = (V, E)$

Output: Partition of V into two parts $(S, V - S)$

Goal: Maximize size of cut $|E(S, V - S)|$



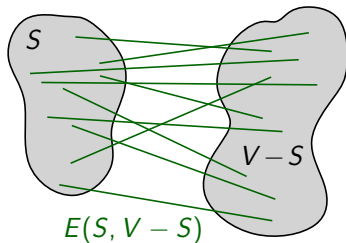
Max-Cut

The MAX-CUT problem:

Input: Graph $G = (V, E)$

Output: Partition of V into two parts $(S, V - S)$

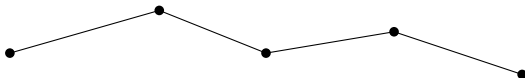
Goal: Maximize size of cut $|E(S, V - S)|$



Poll: have you seen this problem before?

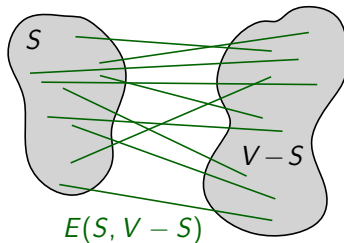
Test yourself: Max-Cut

What is the Max-Cut in the following graph?



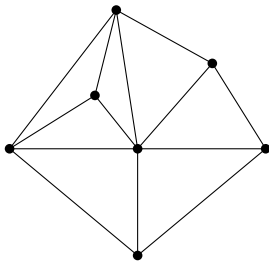
Test yourself: Max-Cut

When is there a “perfect” cut in the graph, which cuts every edge?



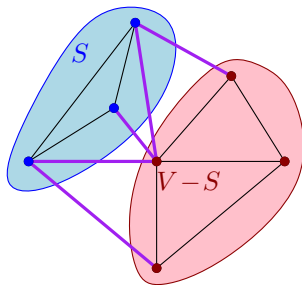
Test yourself: Max-Cut

What is a maximum cut in this graph?



Attempts

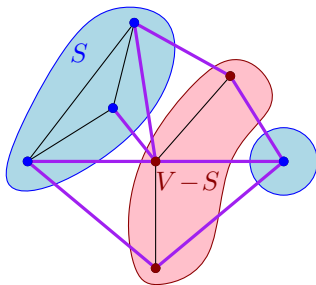
Is this a maximum cut in this graph?



5 edges cut.

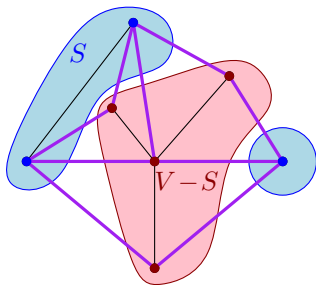
Attempts

Is this a maximum cut in this graph?



Attempts

Is this a maximum cut in this graph?



9 edges cut.

Any ideas?

Now think about an algorithm, say whatever comes to mind.

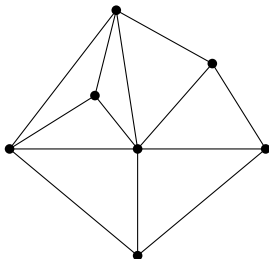
Greedy Max-Cut algorithm

```
1 function GreedyMaxCut( $G$ ):  
2   Let  $S = T = \emptyset$   
3   foreach  $v \in V$  :  
4     if  $|E(v, S)| > |E(v, T)|$  then  $T \leftarrow T \cup \{v\}$   
5     else  $S \leftarrow S \cup \{v\}$   
6   return  $(S, T)$ 
```

Greedy Max-Cut algorithm

```
1 function GreedyMaxCut( $G$ ):  
2   Let  $S = T = \emptyset$   
3   foreach  $v \in V$  :  
4     if  $|E(v, S)| > |E(v, T)|$  then  $T \leftarrow T \cup \{v\}$   
5     else  $S \leftarrow S \cup \{v\}$   
6   return ( $S, T$ )
```

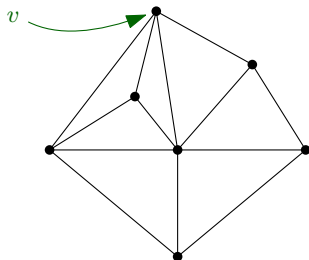
Example run:



Greedy Max-Cut algorithm

```
1 function GreedyMaxCut( $G$ ):  
2   Let  $S = T = \emptyset$   
3   foreach  $v \in V$  :  
4     if  $|E(v, S)| > |E(v, T)|$  then  $T \leftarrow T \cup \{v\}$   
5     else  $S \leftarrow S \cup \{v\}$   
6   return ( $S, T$ )
```

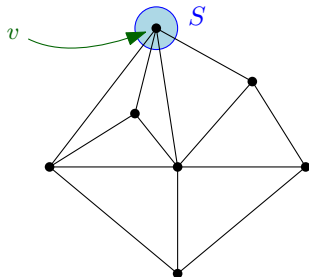
Example run:



Greedy Max-Cut algorithm

```
1 function GreedyMaxCut( $G$ ):  
2   Let  $S = T = \emptyset$   
3   foreach  $v \in V$  :  
4     if  $|E(v, S)| > |E(v, T)|$  then  $T \leftarrow T \cup \{v\}$   
5     else  $S \leftarrow S \cup \{v\}$   
6   return ( $S, T$ )
```

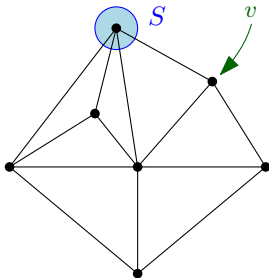
Example run:



Greedy Max-Cut algorithm

```
1 function GreedyMaxCut( $G$ ):  
2   Let  $S = T = \emptyset$   
3   foreach  $v \in V$  :  
4     if  $|E(v, S)| > |E(v, T)|$  then  $T \leftarrow T \cup \{v\}$   
5     else  $S \leftarrow S \cup \{v\}$   
6   return ( $S, T$ )
```

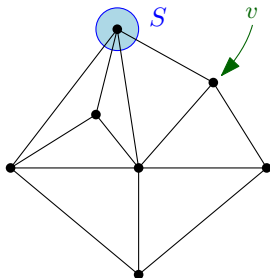
Example run:



Greedy Max-Cut algorithm

```
1 function GreedyMaxCut( $G$ ):  
2   Let  $S = T = \emptyset$   
3   foreach  $v \in V$  :  
4     if  $|E(v, S)| > |E(v, T)|$  then  $T \leftarrow T \cup \{v\}$   
5     else  $S \leftarrow S \cup \{v\}$   
6   return  $(S, T)$ 
```

Example run:



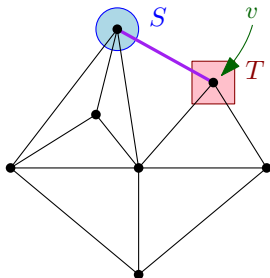
$$|E(v, S)| = 1$$

$$|E(v, T)| = 0$$

Greedy Max-Cut algorithm

```
1 function GreedyMaxCut( $G$ ):  
2   Let  $S = T = \emptyset$   
3   foreach  $v \in V$  :  
4     if  $|E(v, S)| > |E(v, T)|$  then  $T \leftarrow T \cup \{v\}$   
5     else  $S \leftarrow S \cup \{v\}$   
6   return ( $S, T$ )
```

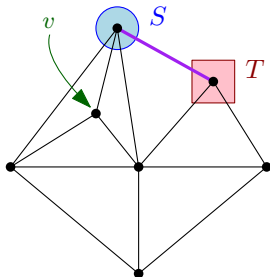
Example run:



Greedy Max-Cut algorithm

```
1 function GreedyMaxCut( $G$ ):  
2   Let  $S = T = \emptyset$   
3   foreach  $v \in V$  :  
4     if  $|E(v, S)| > |E(v, T)|$  then  $T \leftarrow T \cup \{v\}$   
5     else  $S \leftarrow S \cup \{v\}$   
6   return ( $S, T$ )
```

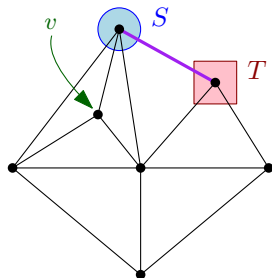
Example run:



Greedy Max-Cut algorithm

```
1 function GreedyMaxCut( $G$ ):  
2   Let  $S = T = \emptyset$   
3   foreach  $v \in V$  :  
4     if  $|E(v, S)| > |E(v, T)|$  then  $T \leftarrow T \cup \{v\}$   
5     else  $S \leftarrow S \cup \{v\}$   
6   return  $(S, T)$ 
```

Example run:



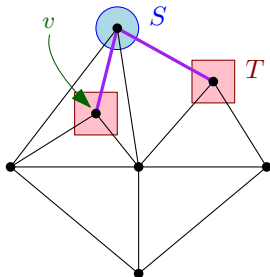
$$|E(v, S)| = 1$$

$$|E(v, T)| = 0$$

Greedy Max-Cut algorithm

```
1 function GreedyMaxCut( $G$ ):  
2   Let  $S = T = \emptyset$   
3   foreach  $v \in V$  :  
4     if  $|E(v, S)| > |E(v, T)|$  then  $T \leftarrow T \cup \{v\}$   
5     else  $S \leftarrow S \cup \{v\}$   
6   return ( $S, T$ )
```

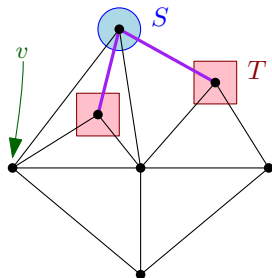
Example run:



Greedy Max-Cut algorithm

```
1 function GreedyMaxCut( $G$ ):  
2   Let  $S = T = \emptyset$   
3   foreach  $v \in V$  :  
4     if  $|E(v, S)| > |E(v, T)|$  then  $T \leftarrow T \cup \{v\}$   
5     else  $S \leftarrow S \cup \{v\}$   
6   return  $(S, T)$ 
```

Example run:



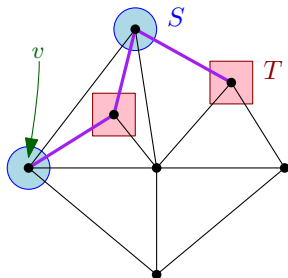
$$|E(v, S)| = 1$$

$$|E(v, T)| = 1$$

Greedy Max-Cut algorithm

```
1 function GreedyMaxCut( $G$ ):  
2   Let  $S = T = \emptyset$   
3   foreach  $v \in V$  :  
4     if  $|E(v, S)| > |E(v, T)|$  then  $T \leftarrow T \cup \{v\}$   
5     else  $S \leftarrow S \cup \{v\}$   
6   return ( $S, T$ )
```

Example run:



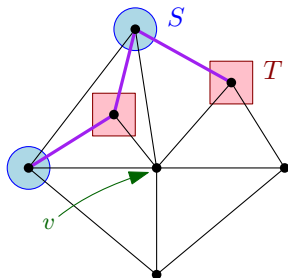
$$|E(v, S)| = 1$$

$$|E(v, T)| = 1$$

Greedy Max-Cut algorithm

```
1 function GreedyMaxCut( $G$ ):  
2   Let  $S = T = \emptyset$   
3   foreach  $v \in V$  :  
4     if  $|E(v, S)| > |E(v, T)|$  then  $T \leftarrow T \cup \{v\}$   
5     else  $S \leftarrow S \cup \{v\}$   
6   return ( $S, T$ )
```

Example run:



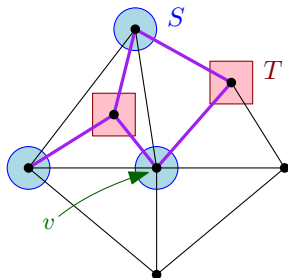
$$|E(v, S)| = 2$$

$$|E(v, T)| = 2$$

Greedy Max-Cut algorithm

```
1 function GreedyMaxCut( $G$ ):  
2   Let  $S = T = \emptyset$   
3   foreach  $v \in V$  :  
4     if  $|E(v, S)| > |E(v, T)|$  then  $T \leftarrow T \cup \{v\}$   
5     else  $S \leftarrow S \cup \{v\}$   
6   return ( $S, T$ )
```

Example run:



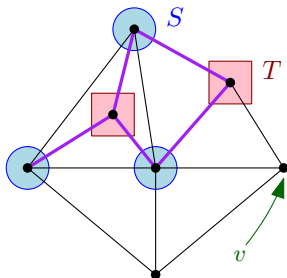
$$|E(v, S)| = 2$$

$$|E(v, T)| = 2$$

Greedy Max-Cut algorithm

```
1 function GreedyMaxCut( $G$ ):  
2   Let  $S = T = \emptyset$   
3   foreach  $v \in V$  :  
4     if  $|E(v, S)| > |E(v, T)|$  then  $T \leftarrow T \cup \{v\}$   
5     else  $S \leftarrow S \cup \{v\}$   
6   return  $(S, T)$ 
```

Example run:



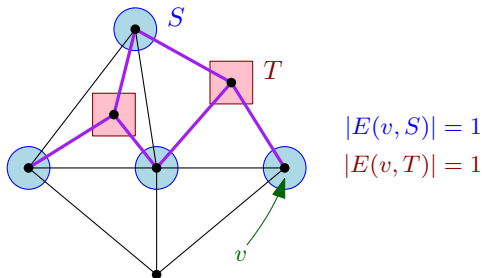
$$|E(v, S)| = 1$$

$$|E(v, T)| = 1$$

Greedy Max-Cut algorithm

```
1 function GreedyMaxCut( $G$ ):  
2   Let  $S = T = \emptyset$   
3   foreach  $v \in V$  :  
4     if  $|E(v, S)| > |E(v, T)|$  then  $T \leftarrow T \cup \{v\}$   
5     else  $S \leftarrow S \cup \{v\}$   
6   return  $(S, T)$ 
```

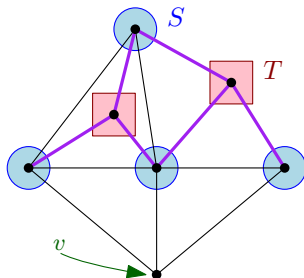
Example run:



Greedy Max-Cut algorithm

```
1 function GreedyMaxCut( $G$ ):  
2   Let  $S = T = \emptyset$   
3   foreach  $v \in V$  :  
4     if  $|E(v, S)| > |E(v, T)|$  then  $T \leftarrow T \cup \{v\}$   
5     else  $S \leftarrow S \cup \{v\}$   
6   return ( $S, T$ )
```

Example run:



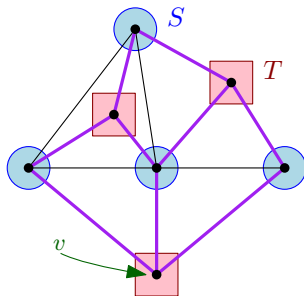
$$|E(v, S)| = 3$$

$$|E(v, T)| = 0$$

Greedy Max-Cut algorithm

```
1 function GreedyMaxCut( $G$ ):  
2   Let  $S = T = \emptyset$   
3   foreach  $v \in V$  :  
4     if  $|E(v, S)| > |E(v, T)|$  then  $T \leftarrow T \cup \{v\}$   
5     else  $S \leftarrow S \cup \{v\}$   
6   return ( $S, T$ )
```

Example run:



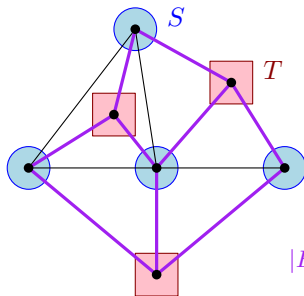
$$|E(v, S)| = 3$$

$$|E(v, T)| = 0$$

Greedy Max-Cut algorithm

```
1 function GreedyMaxCut( $G$ ):  
2   Let  $S = T = \emptyset$   
3   foreach  $v \in V$  :  
4     if  $|E(v, S)| > |E(v, T)|$  then  $T \leftarrow T \cup \{v\}$   
5     else  $S \leftarrow S \cup \{v\}$   
6   return ( $S, T$ )
```

Example run:



Done!
 $|E(S, T)| = 9$

Any ideas?

Time for the analysis. Why would the algorithm cut many edges?

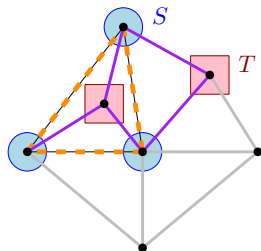
Analysis

During algorithm, three types of edges:

E_{cut} = edges between S and T

E_{uncut} = edges inside either S or T

E_{rem} = remaining edges



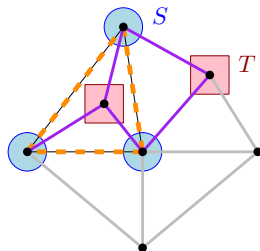
Analysis

During algorithm, three types of edges:

E_{cut} = edges between S and T

E_{uncut} = edges inside either S or T

E_{rem} = remaining edges



Lemma 1.

At every stage of the algorithm $|E_{\text{cut}}| \geq |E_{\text{uncut}}|$.

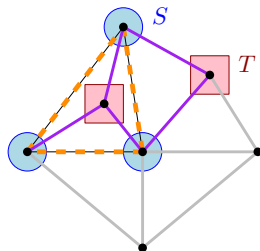
Analysis

During algorithm, three types of edges:

E_{cut} = edges between S and T

E_{uncut} = edges inside either S or T

E_{rem} = remaining edges



Lemma 1.

At every stage of the algorithm $|E_{\text{cut}}| \geq |E_{\text{uncut}}|$.

Proof by induction.

Base case:

Initially $E_{\text{cut}} = E_{\text{uncut}} = \emptyset$ so the claim is true.



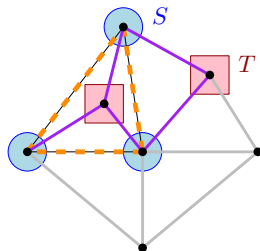
Analysis

During algorithm, three types of edges:

E_{cut} = edges between S and T

E_{uncut} = edges inside either S or T

E_{rem} = remaining edges



Lemma 1.

At every stage of the algorithm $|E_{\text{cut}}| \geq |E_{\text{uncut}}|$.

Proof by induction.

Inductive step: Assume $|E_{\text{cut}}| \geq |E_{\text{uncut}}|$ when we process v .

If $|E(v, S)| > |E(v, T)|$ then we *add v to T* and $|E_{\text{cut}}|$ changes to

$$|E_{\text{cut}}'| = |E_{\text{cut}}| + |E(v, S)| \geq |E_{\text{uncut}}| + |E(v, T)| = |E_{\text{uncut}}'|$$



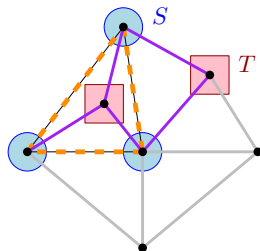
Analysis

During algorithm, three types of edges:

E_{cut} = edges between S and T

E_{uncut} = edges inside either S or T

E_{rem} = remaining edges



Lemma 1.

At every stage of the algorithm $|E_{\text{cut}}| \geq |E_{\text{uncut}}|$.

Proof by induction.

Inductive step: Assume $|E_{\text{cut}}| \geq |E_{\text{uncut}}|$ when we process v .

The case when we add v to S works the same way, we skip it here.



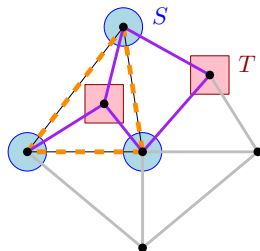
Analysis

During algorithm, three types of edges:

E_{cut} = edges between S and T

E_{uncut} = edges inside either S or T

E_{rem} = remaining edges



Lemma 1.

At every stage of the algorithm $|E_{\text{cut}}| \geq |E_{\text{uncut}}|$.

Note: at end of algorithm, every edge is either in E_{cut} or E_{uncut} .

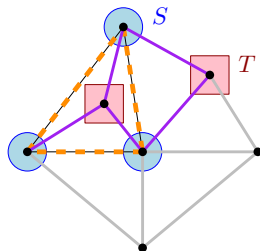
Analysis

During algorithm, three types of edges:

E_{cut} = edges between S and T

E_{uncut} = edges inside either S or T

E_{rem} = remaining edges



Lemma 1.

At every stage of the algorithm $|E_{\text{cut}}| \geq |E_{\text{uncut}}|$.

Note: at end of algorithm, every edge is either in E_{cut} or E_{uncut} .

So Lemma 1 implies that $\mathbf{Alg} = |E_{\text{cut}}| \geq |E|/2$.

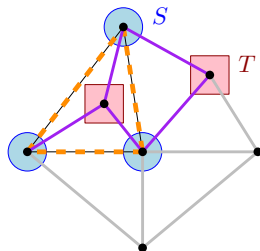
Analysis

During algorithm, three types of edges:

E_{cut} = edges between S and T

E_{uncut} = edges inside either S or T

E_{rem} = remaining edges



Lemma 1.

At every stage of the algorithm $|E_{\text{cut}}| \geq |E_{\text{uncut}}|$.

Note: at end of algorithm, every edge is either in E_{cut} or E_{uncut} .

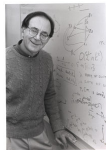
So Lemma 1 implies that $\mathbf{Alg} = |E_{\text{cut}}| \geq |E|/2$.

We also have $\mathbf{Opt} \leq |E|$, so this is a 0.5-approximation-algorithm.

Can we do better?

Brief history of Max-Cut approximation:

- ▶ [Karp'72]: 21 NP-hard problems

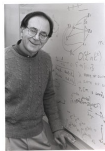


Richard
Karp

Can we do better?

Brief history of Max-Cut approximation:

- ▶ [Karp'72]: 21 NP-hard problems
- ▶ [Goemans-Williamson'92]: can approx within $\alpha_{GW} \approx 0.878$



Richard
Karp



Michel
Goemans



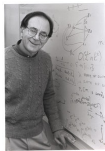
David
Williamson

Can we do better?

Brief history of Max-Cut approximation:

- ▶ [Karp'72]: 21 NP-hard problems
- ▶ [Goemans-Williamson'92]: can approx within $\alpha_{GW} \approx 0.878$

$$\left(\alpha_{GW} = \min_{-1 \leq \rho \leq 1} \frac{2 \arccos \rho}{\pi(1 - \rho)} \right)$$



Richard
Karp



Michel
Goemans



David
Williamson

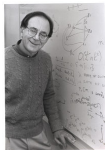
Can we do better?

Brief history of Max-Cut approximation:

- ▶ [Karp'72]: 21 NP-hard problems
- ▶ [Goemans-Williamson'92]: can approx within $\alpha_{GW} \approx 0.878$

$$\left(\alpha_{GW} = \min_{-1 \leq \rho \leq 1} \frac{2 \arccos \rho}{\pi(1 - \rho)} \right)$$

- ▶ [Håstad'01]: NP-hard to approx within $16/17 + \epsilon \approx 0.941$



Richard
Karp



Michel
Goemans



David
Williamson



Johan
Håstad

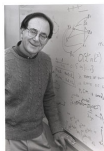
Can we do better?

Brief history of Max-Cut approximation:

- ▶ [Karp'72]: 21 NP-hard problems
- ▶ [Goemans-Williamson'92]: can approx within $\alpha_{GW} \approx 0.878$

$$\left(\alpha_{GW} = \min_{-1 \leq \rho \leq 1} \frac{2 \arccos \rho}{\pi(1 - \rho)} \right)$$

- ▶ [Håstad'01]: NP-hard to approx within $16/17 + \varepsilon \approx 0.941$
- ▶ [Khot, Kindler, Mossel, O'Donnell'04]: cannot get $\alpha_{GW} + \varepsilon$ assuming Unique Games Conjecture.



Richard
Karp



Michel
Goemans



David
Williamson



Johan
Håstad



Subhash
Khot



Guy
Kindler



Elchanan
Mossel



Ryan
O'Donnell

Agenda

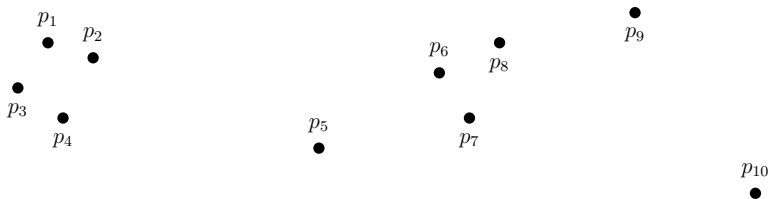
Vertex Cover

Max-Cut

k -Center

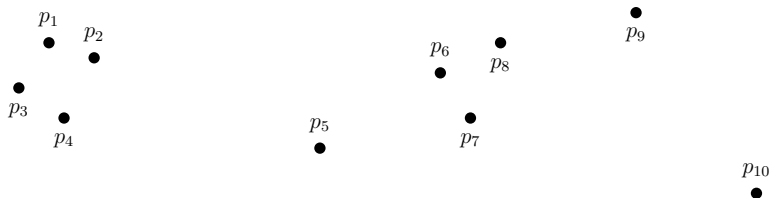
Clustering

We have n points that we want to group into k clusters



Clustering

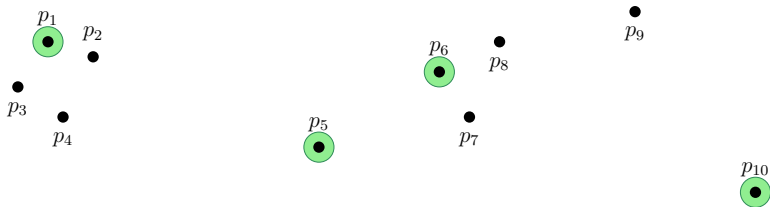
We have n points that we want to group into k clusters



Want to pick set S of k of the points as centers.

Clustering

We have n points that we want to group into k clusters

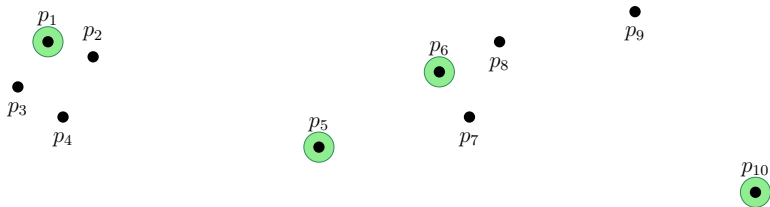


Want to pick set S of k of the points as centers.

$$S = \{p_1, p_5, p_6, p_{10}\}$$

Clustering

We have n points that we want to group into k clusters



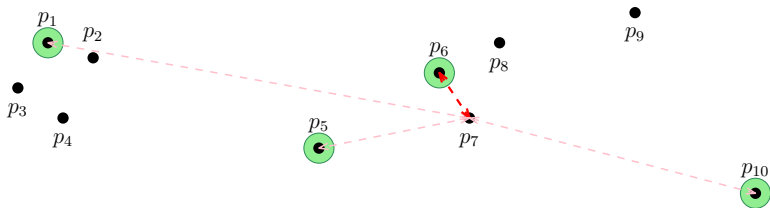
Want to pick set S of k of the points as centers.

$$S = \{p_1, p_5, p_6, p_{10}\}$$

Distance $d(p, S)$ is distance from p to closest point in S .

Clustering

We have n points that we want to group into k clusters



Want to pick set S of k of the points as centers.

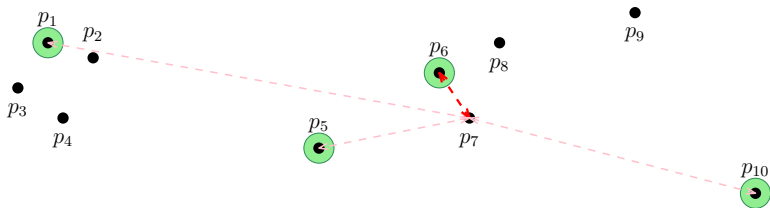
$$S = \{p_1, p_5, p_6, p_{10}\}$$

Distance $d(p, S)$ is distance from p to closest point in S .

$$d(p_7, S) = \min(d(p_7, p_1), d(p_7, p_5), d(p_7, p_6), d(p_7, p_{10})) = d(p_7, p_6)$$

Clustering

We have n points that we want to group into k clusters



Want to pick set S of k of the points as centers.

$$S = \{p_1, p_5, p_6, p_{10}\}$$

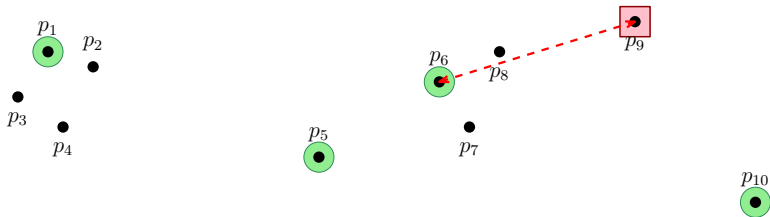
Distance $d(p, S)$ is distance from p to closest point in S .

$$d(p_7, S) = \min(d(p_7, p_1), d(p_7, p_5), d(p_7, p_6), d(p_7, p_{10})) = d(p_7, p_6)$$

Want to choose S such that furthest point from S is as close as possible.

Clustering

We have n points that we want to group into k clusters



Want to pick set S of k of the points as centers.

$$S = \{p_1, p_5, p_6, p_{10}\}$$

Distance $d(p, S)$ is distance from p to closest point in S .

$$d(p_7, S) = \min(d(p_7, p_1), d(p_7, p_5), d(p_7, p_6), d(p_7, p_{10})) = d(p_7, p_6)$$

Want to choose S such that furthest point from S is as close as possible.

p_9 is furthest point, at distance $d(p_9, p_6)$

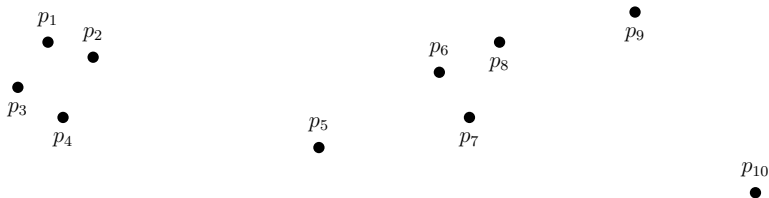
The k -Center Problem

k -center:

Input: n points $p_1, \dots, p_n$, and a size k .

Output: Set $S \subseteq \{p_1, \dots, p_n\}$ of k points

Goal: Minimize the maximum distance $\max_{i=1..n} d(p_i, S)$



The k -Center Problem

k -center:

Input: n points $p_1, \dots, p_n$, and a size k .

Output: Set $S \subseteq \{p_1, \dots, p_n\}$ of k points

Goal: Minimize the maximum distance $\max_{i=1..n} d(p_i, S)$

Poll: have you seen this problem before?

Some context

k -center:

Input: n points $p_1, \dots, p_n$, and a size k .

Output: Set $S \subseteq \{p_1, \dots, p_n\}$ of k points

Goal: Minimize the maximum distance $\max_{i=1..n} d(p_i, S)$

- ▶ sometimes you don't have to pick S from the set of points given (makes a big difference - which version do you think is the hardest?)
- ▶ the objective function can vary, some examples
 - ▶ *k -median*: min (average) **sum** of dist $\sum_{p_i} d(p_i, \text{closest center})$
 - ▶ *k -means*: min sum of **squared** dist $\sum_{p_i} (d(p_i, \text{closest center}))^2$
 - ▶ *facility location*: there are **opening costs**, w_i = price you pay for opening a center at p_i , and objective is to min
cost of opening the centers + k -median objective

Some more context

k -center:

Input: n points $p_1, \dots, p_n$, and a size k .

Output: Set $S \subseteq \{p_1, \dots, p_n\}$ of k points

Goal: Minimize the maximum distance $\max_{i=1..n} d(p_i, S)$

Can you think of other variants of the problem? Add more constraints to make the solution more "meaningful"....

Some more context

k -center:

Input: n points $p_1, \dots, p_n$, and a size k .

Output: Set $S \subseteq \{p_1, \dots, p_n\}$ of k points

Goal: Minimize the maximum distance $\max_{i=1..n} d(p_i, S)$

Can you think of other variants of the problem? Add more constraints to make the solution more "meaningful"....

- ▶ add capacities, e.g., each cluster can hold at most 10 nodes
- ▶ allow outliers, i.e., smaller clusters
- ▶ impose fairness constraints
- ▶ what if the points change over time?

Any ideas?

Now think about an algorithm, say whatever comes to mind.

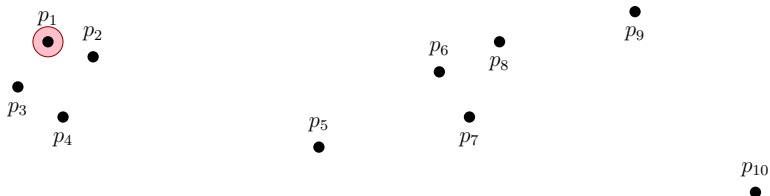
Greedy Algorithm

```
1 function kCenter( $k, p_1, \dots, p_n$ ):  
2   Let  $S = \{p_1\}$   
3   while  $|S| < k$   
4     Let  $i$  be such that  $d(p_i, S)$  is maximized  
5      $S \leftarrow S \cup \{p_i\}$   
6   return  $S$ 
```

Greedy Algorithm

```
1 function kCenter( $k, p_1, \dots, p_n$ ):  
2   Let  $S = \{p_1\}$   
3   while  $|S| < k$   
4     Let  $i$  be such that  $d(p_i, S)$  is maximized  
5      $S \leftarrow S \cup \{p_i\}$   
6   return  $S$ 
```

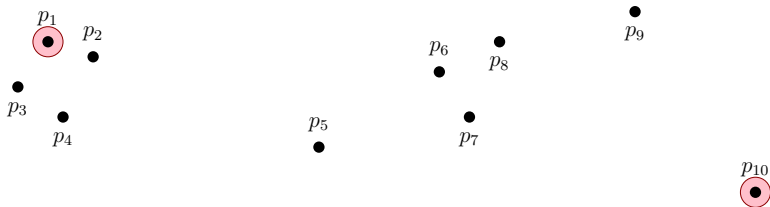
Example run ($k = 4$):



Greedy Algorithm

```
1 function kCenter( $k, p_1, \dots, p_n$ ):  
2   Let  $S = \{p_1\}$   
3   while  $|S| < k$   
4     Let  $i$  be such that  $d(p_i, S)$  is maximized  
5      $S \leftarrow S \cup \{p_i\}$   
6   return  $S$ 
```

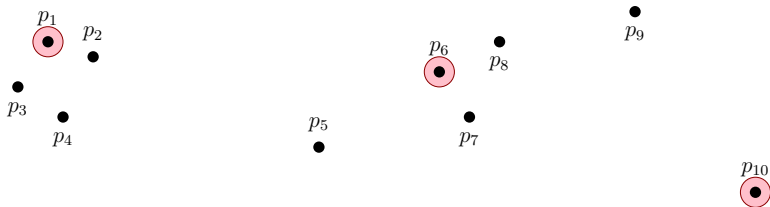
Example run ($k = 4$):



Greedy Algorithm

```
1 function kCenter( $k, p_1, \dots, p_n$ ):  
2   Let  $S = \{p_1\}$   
3   while  $|S| < k$   
4     Let  $i$  be such that  $d(p_i, S)$  is maximized  
5      $S \leftarrow S \cup \{p_i\}$   
6   return  $S$ 
```

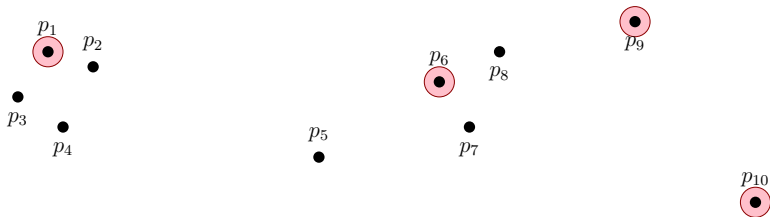
Example run ($k = 4$):



Greedy Algorithm

```
1 function kCenter( $k, p_1, \dots, p_n$ ):  
2   Let  $S = \{p_1\}$   
3   while  $|S| < k$   
4     Let  $i$  be such that  $d(p_i, S)$  is maximized  
5      $S \leftarrow S \cup \{p_i\}$   
6   return  $S$ 
```

Example run ($k = 4$):

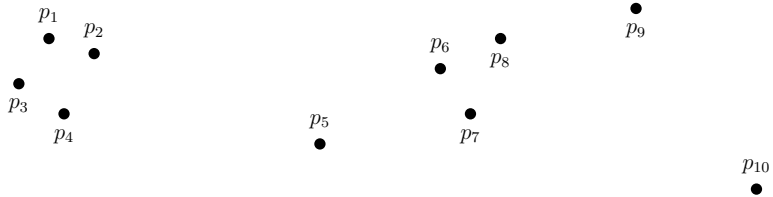


Greedy Algorithm

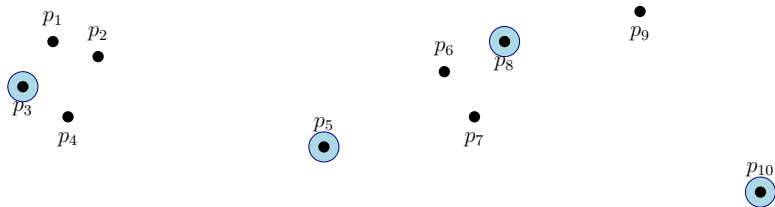
```
1 function kCenter( $k, p_1, \dots, p_n$ ):  
2   Let  $S = \{p_1\}$   
3   while  $|S| < k$   
4     Let  $i$  be such that  $d(p_i, S)$  is maximized  
5      $S \leftarrow S \cup \{p_i\}$   
6   return  $S$ 
```

How to analyze this algorithm?

Analysis

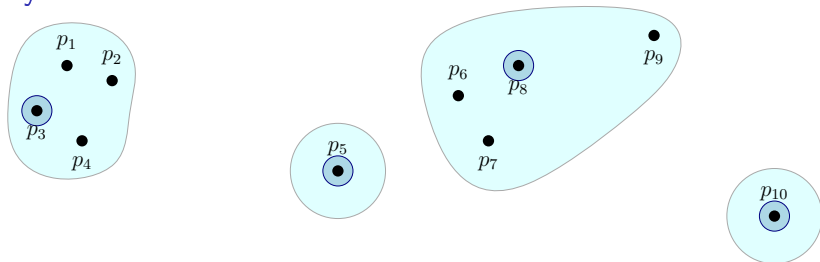


Analysis



Let S_{Opt} be the optimal set of points.

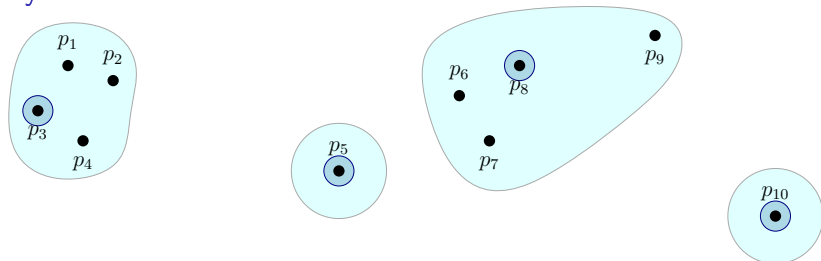
Analysis



Let S_{Opt} be the optimal set of points.

S_{Opt} partitions the points into k clusters.

Analysis

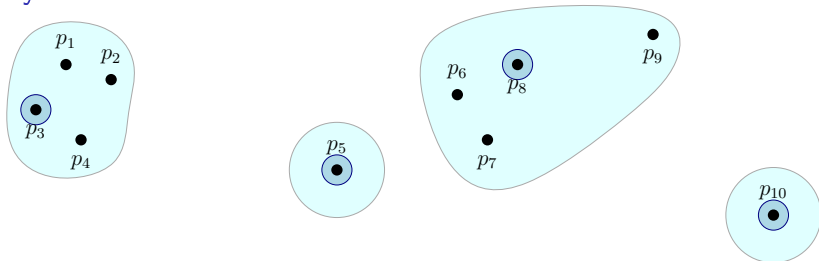


Let S_{Opt} be the optimal set of points.

S_{Opt} partitions the points into k clusters.

p_i belongs to cluster x if x is the point in S_{Opt} closest to p_i

Analysis

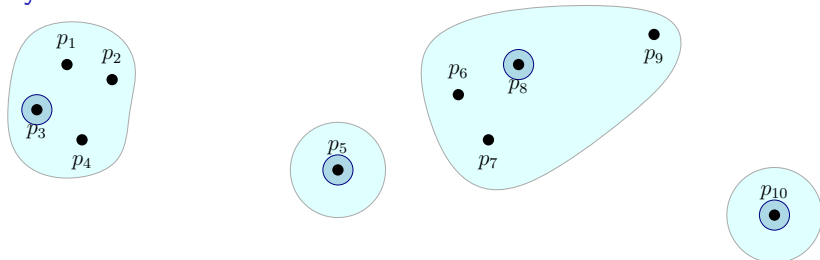


Key property of the clusters:

Lemma 1.

If p_i and p_j are in the same cluster in S_{Opt} , then $d(p_i, p_j) \leq 2Opt$.

Analysis



Key property of the clusters:

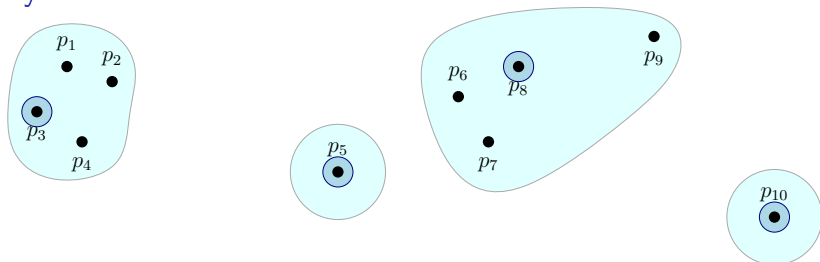
Lemma 1.

If p_i and p_j are in the same cluster in S_{Opt} , then $d(p_i, p_j) \leq 2\text{Opt}$.

Proof.

If p_i and p_j belong to the cluster of some $x \in S_{\text{Opt}}$ then $d(p_i, x) \leq \text{Opt}$ and $d(p_j, x) \leq \text{Opt}$. So by the triangle inequality, $d(p_i, p_j) \leq d(p_i, x) + d(x, p_j) \leq 2\text{Opt}$. □

Analysis



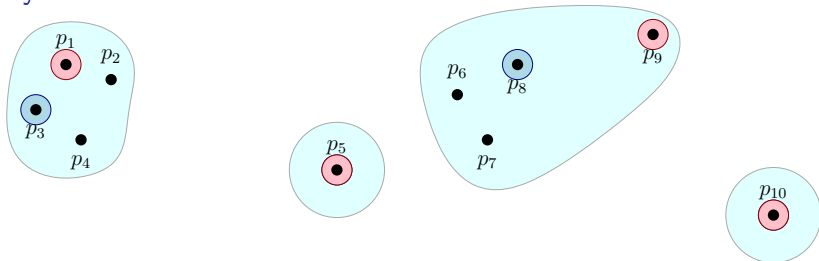
How are points we choose in the algorithm spread across the clusters in S_{Opt} ?

Two cases:

Case 1: We pick one point p_i from each cluster in S_{Opt} .

Case 2: We pick two points p_i and p_j from the same cluster in S_{Opt} .

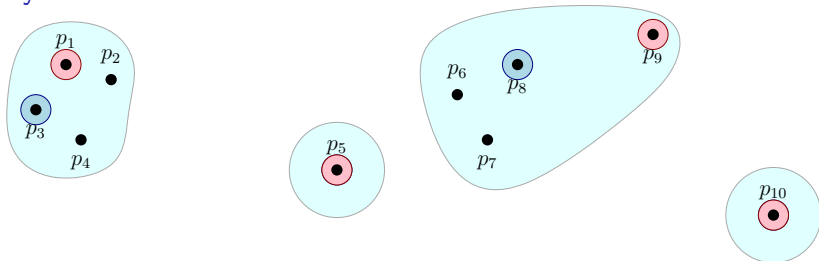
Analysis



Lemma 2.

If algorithm picks one point p_i from each cluster then $\mathbf{Alg} \leq 2\mathbf{Opt}$.

Analysis



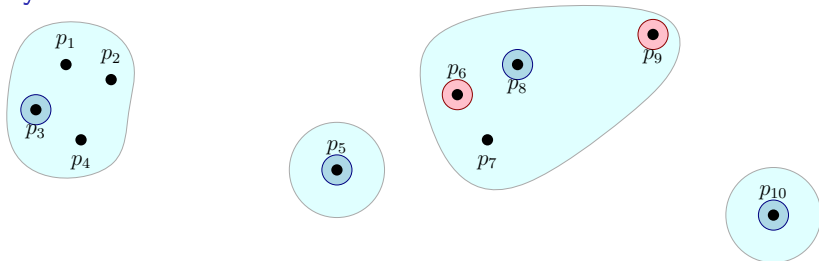
Lemma 2.

If algorithm picks one point p_i from each cluster then $\mathbf{Alg} \leq 2\mathbf{Opt}$.

Proof.

Consider the cluster of any $x \in S_{\mathbf{Opt}}$. Let p_i be the point chosen by the algorithm in this cluster. For any point p_j in the cluster we have $d(p_i, p_j) \leq 2\mathbf{Opt}$ by Lemma 1. □

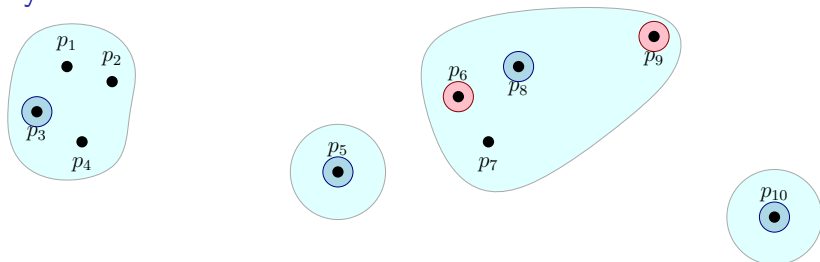
Analysis



Lemma 3.

If algorithm picks both p_i and p_j from same cluster then $Alg \leq d(p_i, p_j)$.

Analysis



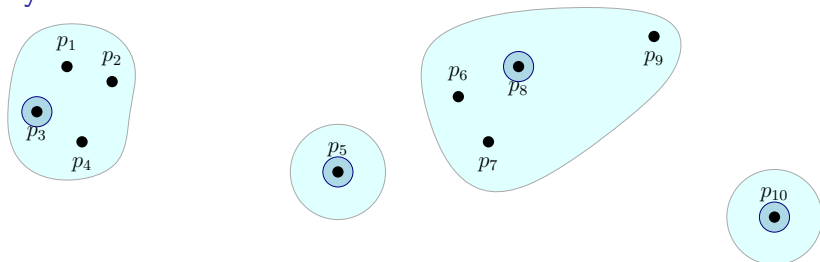
Lemma 3.

If algorithm picks both p_i and p_j from same cluster then $\text{Alg} \leq d(p_i, p_j)$.

Proof.

Suppose p_i picked first then p_j . When p_j is picked, it is the point that is furthest away from all previously picked points. So we have $\text{Alg} = \max_{k=1..n} d(p_k, S_{\text{Alg}}) \leq d(p_i, p_j)$ □

Analysis



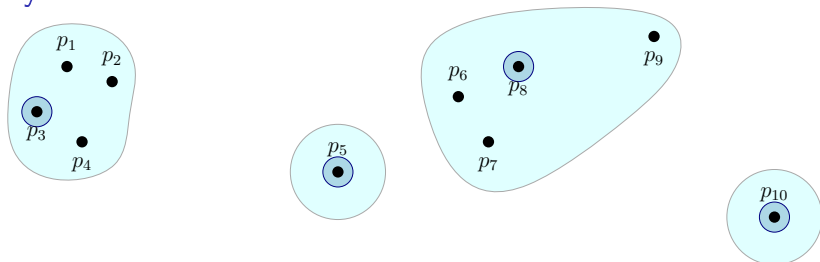
How are points we choose in the algorithm spread across the clusters in S_{Opt} ?

Two cases:

Case 1: We pick one point p_i from each cluster in S_{Opt} .

Case 2: We pick two points p_i and p_j from the same cluster in S_{Opt} .

Analysis



How are points we choose in the algorithm spread across the clusters in S_{Opt} ?

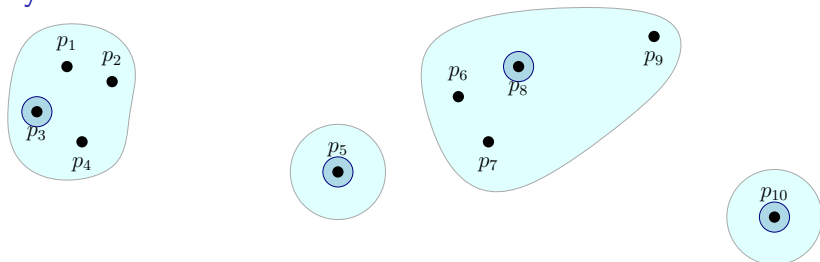
Two cases:

Case 1: We pick one point p_i from each cluster in S_{Opt} .

Lemma 3 $\Rightarrow \text{Alg} \leq 2\text{Opt}$

Case 2: We pick two points p_i and p_j from the same cluster in S_{Opt} .

Analysis



How are points we choose in the algorithm spread across the clusters in S_{Opt} ?

Two cases:

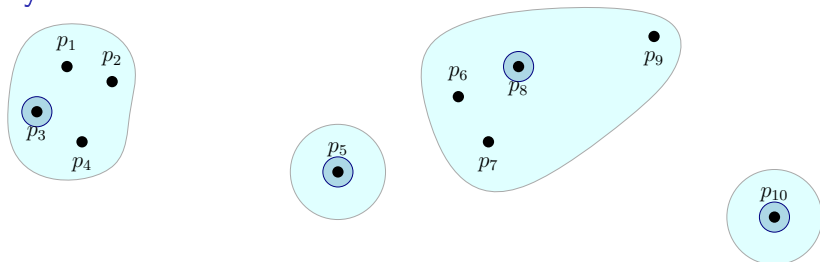
Case 1: We pick one point p_i from each cluster in S_{Opt} .

Lemma 3 $\Rightarrow \mathbf{Alg} \leq 2\mathbf{Opt}$

Case 2: We pick two points p_i and p_j from the same cluster in S_{Opt} .

Lemma 2 $\Rightarrow \mathbf{Alg} \leq d(p_i, p_j) \leq 2\mathbf{Opt}$

Analysis



How are points we choose in the algorithm spread across the clusters in S_{Opt} ?

Two cases:

Case 1: We pick one point p_i from each cluster in S_{Opt} .

Lemma 3 $\Rightarrow \mathbf{Alg} \leq 2\mathbf{Opt}$

Case 2: We pick two points p_i and p_j from the same cluster in S_{Opt} .

Lemma 2 $\Rightarrow \mathbf{Alg} \leq d(p_i, p_j) \leq 2\mathbf{Opt}$

General Metrics

What did we use in the analysis?

Is it important that the points are in $2D$ with Euclidean distances?

General Metrics

What did we use in the analysis?

Is it important that the points are in $2D$ with Euclidean distances?

The only thing used was that we have a **metric space**:

There is a distance function $d(p, q)$ between points such that

1. $d(p, p) = 0$
2. If $p \neq q$ then $d(p, q) > 0$ (positivity)
3. $d(p, q) = d(q, p)$ (symmetric)
4. $d(p, q) \leq d(p, r) + d(r, q)$ (triangle inequality)

Alternate algorithms

Suppose we know the value of **Opt** (the optimal radius). Can we use this extra information?

Alternate algorithms

Suppose we know the value of **Opt** (the optimal radius). Can we use this extra information?

Yes! Pick a center and remove everything around it within distance.... **WHAT?**

Alternate algorithms

Suppose we know the value of **Opt** (the optimal radius). Can we use this extra information?

Yes! Pick a center and remove everything around it within distance.... **WHAT?**

Why would this work? Why is it enough to take balls of radius **2Opt** (and not **3Opt**)?

Alternate algorithms

Suppose we know the value of **Opt** (the optimal radius). **Can we use this extra information?**

Yes! Pick a center and remove everything around it within distance.... **WHAT?**

Why would this work? Why is it enough to take balls of radius **2Opt** (and not **3Opt**)? Try **2Opt**.

Alternate algorithms

Suppose we know the value of **Opt** (the optimal radius). Can we use this extra information?

Yes! Pick a center and remove everything around it within distance.... **WHAT?**

Why would this work? Why is it enough to take balls of radius **2Opt** (and not **3Opt**)? Try **2Opt**.

Now fix the assumption that you knew **Opt**.

Can We Do Better?

For general metrics it is NP-hard to approximate within any factor smaller than 2.

For points in Euclidean 2D space less clear, known to be NP-hard to approximate within 1.93.

Recap

- ▶ we saw approximation algorithms for Vertex Cover, Max Cut and k -center
 - ▶ Vertex Cover: for each uncovered edge, pick both vertices
 - ▶ Max Cut: add a vertex to the partition that increases the number of cut edges
 - ▶ k -center: pick the center that is the farthest away
- ▶ show that **Opt** cannot do **much** better
 - ▶ i.e., whatever choice you make, **Opt** will also have to make a choice
 - ▶ maybe a smarter choice, but not **much** smarter, i.e., the **Opt** value cannot be much bigger/smaller

Next week:

- ▶ next Wednesday 15:00-17:00: Lecture 3
FPT algorithms
- ▶ Get Prepared (no grade) assignment due